~ Complete Step by Step ~

Node.js

Server-side JavaScript • Express • APIs • Diagrams

Index

1 Introduction to Node.js
14 Templates, Forms & File Upload
2 Install & First Steps
15 MongoDB & Mongoose
3 Architecture & Event Loop
16 SQL Databases & ORMs
4 Modules
17 Authentication (JWT, Sessions)
5 npm & package.json
18 Security & Environment
6 Core Modules & Events
19 Real-time: WebSockets
7 File System (fs)
20 Project Structure (MVC)
8 Buffers & Streams
21 Testing & Debugging
9 Async Programming
22 Scaling & Performance
10 HTTP Module
23 Deployment
11 Express.js Basics
24 Mini Project: REST API
12 Middleware & Errors
25 Interview Q&A
13 REST API & CRUD
26 Cheat Sheet

How to use

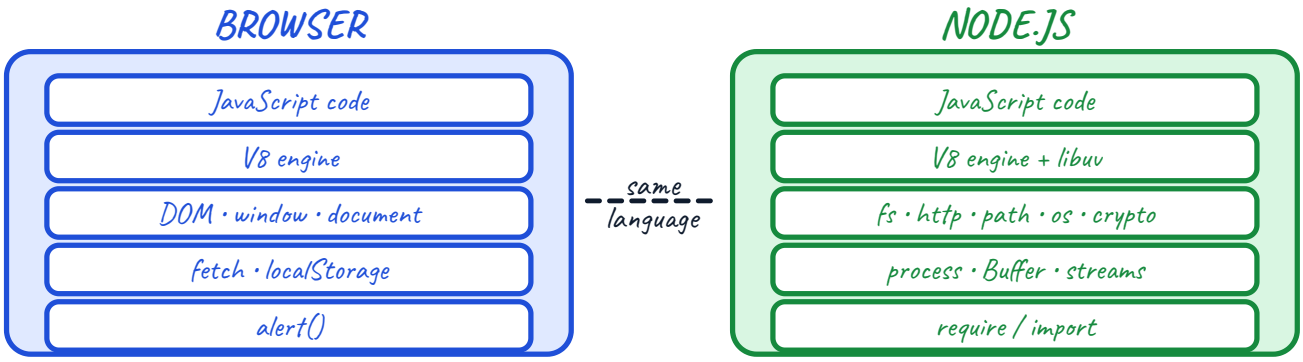
- Learn **JavaScript first** (functions, arrays, objects, promises, async/await, modules).
- Type every **example** yourself in VS Code and run it with ***node file.js***. Redraw the **diagrams** in your notebook.
- Written for **Node.js 20/22 LTS**. Test APIs with **Postman** or **Thunder Client**.

Notes by asadokz18 ✿

1. Introduction to Node.js

Definition

Node.js is a free, open-source, cross-platform **JavaScript runtime environment** that lets you run JavaScript **outside the browser** – on servers, laptops and tools. Created by **Ryan Dahl in 2009**; built on Chrome's **V8 engine** (C++).



Node = JavaScript outside the browser (servers, tools, CLIs, APIs)

Why Node.js?

- → **Fast** – V8 compiles JS to machine code
- → **Non-blocking, event-driven** I/O – handles thousands of connections
- → **One language** (JavaScript) for front-end & back-end
- → **npm** – the largest package ecosystem
- → **Single-threaded** event loop (simple, no locks)
- → **Scalable** – microservices, clusters
- → **Cross-platform** – Windows, macOS, Linux
- → Great for **JSON APIs** & real-time apps

What can you build?

Use case	Examples / tools
REST / GraphQL APIs	Express, Fastify, NestJS
Real-time apps	Chat, live notifications, games (Socket.IO)
Websites / SSR	Next.js, EJS templates
CLI tools & scripts	Vite, ESLint, Prettier, npm itself
Microservices & serverless	AWS Lambda, Vercel functions
Streaming / IoT / desktop	Netflix, Electron (VS Code, Discord)

Used by: Netflix PayPal, LinkedIn, Uber, NASA, Walmart.

Good for

I/O-heavy work: APIs, databases, file & network operations, real-time data.

Not ideal for

Heavy **CPU** tasks (video encoding, ML) on the main thread – it blocks everything. Use worker threads or another language.

Node vs browser JS

No ***window / document / DOM*** in Node; instead you get ***global***, ***process***, ***require***, ***__dirname***, and modules like ***fs*** and ***http***. **LTS** (even versions: 20, 22) = stable for production; **Current** (odd) = newest features.

2. Installation & First Steps

1. Download the **LTS** installer from **nodejs.org** (includes **npm**). 2. Verify in the terminal:

```
node -v          # v22.x.x          npm -v      # 10.x.x
nvm install --lts   &&   nvm use 22        # nvm / fnm = switch Node versions (recommended)
```

Your first program

hello.js

```
console.log("Hello Node.js from asadokz18!");
const name = "Asad";
console.log(`Welcome, ${name}`);          // run:  node hello.js
```

REPL – interactive shell

Type **node** (no file) → Read Eval Print Loop: **> 2 + 3 → 5**. Commands: **.help · .editor · .save f.js · .exit** (or Ctrl+C twice). _ = last result.

Global objects & variables

Global	Meaning
console	log, error, warn, table, time/timeEnd, dir
__dirname / __filename	Absolute folder / file path of the current module (CommonJS)
process	Info & control of the running program
global / globalThis	The global object (like window in browser)
setTimeout / setInterval / setImmediate	Timers (+ clearTimeout …)
Buffer, URL, fetch, structuredClone	Binary data, URL class, built-in fetch (v18+)
require, module, exports	Module system (CommonJS only)

process object – very useful

```
console.log(process.version, process.platform);          // v22.1.0  win32/linux/darwin
console.log(process.argv);                               // [nodePath, scriptPath, ...your args]
console.log(process.env.PORT, process.env.NODE_ENV);     // environment variables
console.log(process.cwd(), process.pid, process.uptime());
console.log(process.memoryUsage().heapUsed);
process.on("exit", (code) => console.log("bye", code));   // events: exit, uncaughtException, SIGINT
process.exit(0);                                         // 0 = success, 1 = error
// node app.js Asad 21   →   const [,, name, age] = process.argv;
```

console methods

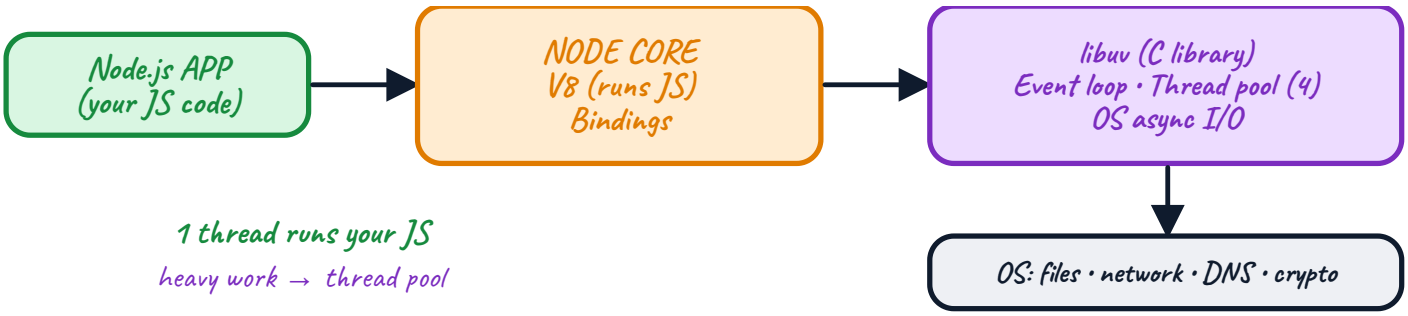
```
console.log("text", obj);
console.error("Oops");  console.warn("hmm");
console.table([{a:1},{a:2}]);
console.time("t"); /* work */ console.timeEnd("t");
console.group("Group"); console.log("in");
console.groupEnd();
```

Handy commands

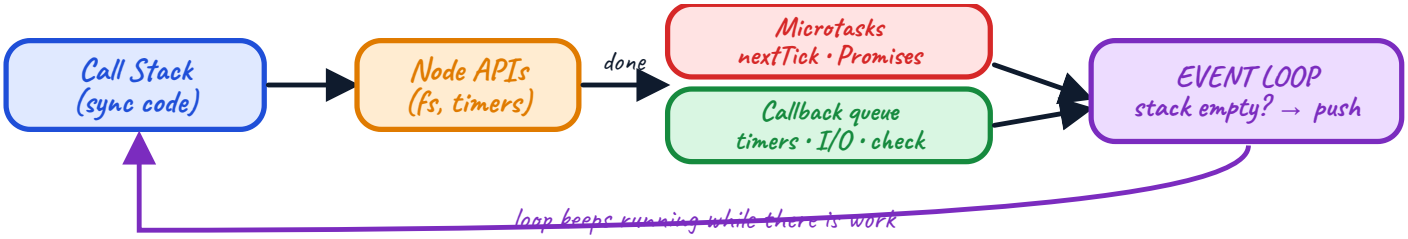
- → **node --watch app.js** auto-restart (v18+)
- → **node --env-file=.env app.js** load .env (v20+)
- → **node -e "console.log(1)"** run inline code
- → **nodemon app.js** (dev tool)

Read input: ***readline*** module (***rl.question("Name? ", cb)***). **Top-level await** works in ES modules. Run in VS Code: Ctrl+` terminal.

3. Architecture & the Event Loop

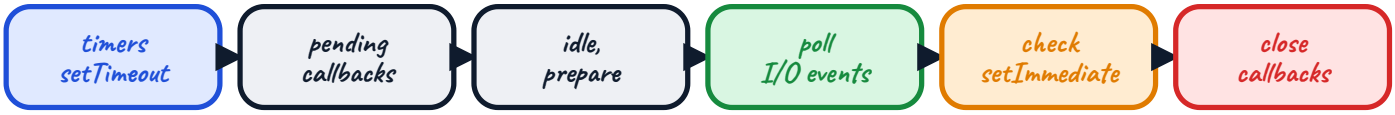


- Node runs your JS on **one main thread**. To stay fast it never **waits**: slow operations (files, network, DB) are handed to the **OS / libuv thread pool**; when done, their **callback** is queued and the **event loop** runs it.
- **Blocking** = the thread waits (e.g. ***fs.readFileSync***) → other requests wait. **Non-blocking** = start the job, continue, get notified later.



Order: sync code → process.nextTick → Promises (microtasks) → timers → I/O → setImmediate

Event loop phases (libuv)



between every phase: run process.nextTick queue, then Promise microtasks

Predict the output – classic interview question

```
console.log("1 start");
setTimeout(() => console.log("5 setTimeout"), 0);        // timers phase
setImmediate(() => console.log("6 setImmediate"));       // check phase
Promise.resolve().then(() => console.log("4 promise"));  // microtask
process.nextTick(() => console.log("3 nextTick"));       // runs before promises
console.log("2 end");
// output order: 1 start → 2 end → 3 nextTick → 4 promise → 5 setTimeout → 6 setImmediate
```

Blocking (sync)	Non-blocking (async)
fs.readFileSync(f)	**fs.readFile(f, cb) / await fs.promises.readFile(f)**
Next line waits	Next line runs immediately; callback later
OK for startup scripts	Required inside servers

Never block the loop

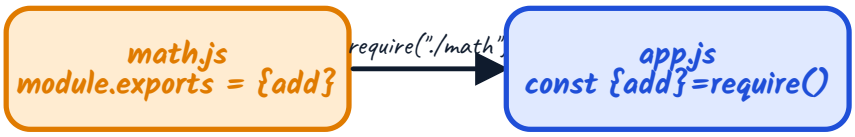
A long **for** loop, ***JSON.parse*** of a huge file or sync crypto freezes **every** user. Move CPU work to **worker threads** / child process.

Thread pool

libuv has **4 threads** by default (***UV_THREADPOOL_SIZE***) for **fs**, **crypto**, **zlib**, DNS lookup. Network I/O uses the OS (epoll/kqueue) – no threads needed.

4. Modules

A **module** is a file with reusable code. Modules keep code organised, avoid global pollution and let you share functions between files. Three kinds: **built-in** (fs), **local** (your files), **third-party** (npm).



CommonJS (default) require / module.exports
ES Modules import / export ("type":"module")
Node caches a module after the 1st load

CommonJS (CJS) – default in Node

math.js

```
const add = (a, b) => a + b;
const PI = 3.14159;
module.exports = { add, PI };            // export many  (or  module.exports = add;  → export one)
// exports.sub = (a,b) => a-b;           // shortcut (don't reassign "exports")
```

app.js

```
const { add, PI } = require("./math");   // local file (./ path, no extension needed)
const fs = require("fs");                // built-in  (also "node:fs")
const express = require("express");      // from node_modules
console.log(add(2, 3), PI);
```

ES Modules (ESM) – modern standard

Enable with **"type": "module"** in package.json (or use **.mjs**):

utils.js

```
export const add = (a, b) => a + b;
export default function greet(n) {
  return `Hi ${n}`;
}
```

main.js

```
import greet, { add } from "./utils.js";  // .js required!
import fs from "node:fs";
import * as utils from "./utils.js";
const data = await import("./x.js");      // dynamic import
```

	CommonJS	ES Modules
Syntax	***require() / module.exports***	***import / export***
Loading	Synchronous	Asynchronous, static analysis (tree-shaking)
__dirname	Available	Not available → ***import.meta.dirname*** (v20.11+) or ***fileURLToPath(import.meta.url)***
Top-level await	No	Yes
Default in	.js (no type) / .cjs	.mjs / type: module

- **Module wrapper**: Node wraps each file in a function ***(exports, require, module, __filename, __dirname) => {...}*** → variables are **private** to the file.
- **Caching**: a module runs once; later ***require*** returns the same object (singleton - great for DB connection).
- **Resolution order**: built-in → ***./relative*** file/folder (***index.js***) → ***node_modules*** up the folders. **Circular** requires give partial exports - avoid.

Which one?

New projects: **ESM**. Older tutorials & many packages: **CJS**. Don't mix ***require*** and ***import*** in one file.

5. npm & package.json

npm (Node Package Manager) installs & manages libraries from the **npm registry**. Alternatives: **yarn**, **pnpm**, **bun**.



Start a project

```
npm init -y                    # creates package.json with defaults
npm install express            # (npm i) install + add to "dependencies"
npm i -D nodemon                 # devDependency (only for development)
npm i -g typescript            # global tool       npm uninstall express   # remove
npm update · npm outdated · npm list --depth=0 · npm audit [fix]      # maintain / security
npx create-vite@latest         # run a package without installing globally
npm ci                         # clean install from package-lock (CI/servers)
```

package.json

```
{
  "name": "my-api",  "version": "1.0.0",  "description": "My first API",
  "main": "index.js",  "type": "module",                  // or "commonjs"
  "scripts": {
    "start": "node index.js",                          // npm start
    "dev": "nodemon index.js",                          // npm run dev
    "test": "jest"                                     // npm test
  },
  "dependencies": { "express": "^4.19.2" },
  "devDependencies": { "nodemon": "^3.1.0" },
  "engines": { "node": ">=20" },  "license": "MIT"
}
```

Version	Meaning (semver MAJOR.MINOR.PATCH)
^4.19.2	Accept **minor + patch** updates (4.x.x) – default
~4.19.2	Accept only **patch** updates (4.19.x)
4.19.2	**Exact** version
MAJOR	Breaking change · **MINOR** new feature · **PATCH** bug fix

- **node_modules/** = downloaded code (huge!) → **never commit**; add to ***.gitignore***. Recreate with ***npm install***.
- **package-lock.json** = exact tree → **commit it** for repeatable installs.
- **dependencies** = needed in production; **devDependencies** = tools (nodemon, jest, eslint); **peerDependencies** = host must provide.
- Scripts: ***npm run build***; ***start*** & ***test*** don't need ***run***. Pre/post hooks: ***prestart***.

Popular packages

express (web) · **mongoose** (MongoDB) · **dotenv** · **cors** · **bcrypt** · **jsonwebtoken** · **axios** · **nodemon** · **multer** · **zod** · **socket.io** · **jest** · **lodash** · **winston/pino** (logs). Publish your own: ***npm login*** → ***npm publish***.

6. Core Modules: path, os, url, util & events

path – work with file paths safely

```
const path = require("path");
path.join(__dirname, "public", "img", "a.png");       // joins with correct separator (/ or \)
path.resolve("src", "app.js");                        // absolute path
path.basename("/a/b/file.txt");  // "file.txt"       path.basename(p, ".txt") → "file"
path.extname("file.txt");        // ".txt"           path.dirname("/a/b/file.txt");  // "/a/b"
path.parse("/a/b/file.txt");     // { root, dir, base, ext, name }
```

os

```
const os = require("os");
os.platform();  os.arch();  os.hostname();
os.cpus().length;              // CPU cores
os.totalmem(); os.freemem();   // bytes
os.homedir(); os.tmpdir(); os.uptime();
```

url & querystring

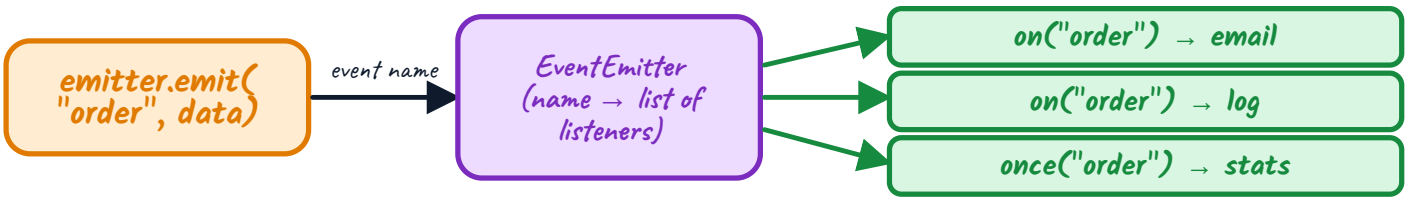
```
const u = new URL("https://x.com/p?id=5&q=js#top");
u.hostname; u.pathname; u.hash;
u.searchParams.get("id");       // "5"
u.searchParams.append("page", 2);
```

util & crypto & others

```
const util = require("util");
const readFile = util.promisify(fs.readFile);         // callback → promise
util.format("%s is %d", "Asad", 21);  util.inspect(obj, { depth: null, colors: true });
const crypto = require("crypto");
crypto.randomUUID();  crypto.randomBytes(16).toString("hex");
crypto.createHash("sha256").update("pass").digest("hex");       // hashing (use bcrypt for passwords!)
```

Module	Purpose
fs / fs/promises	Files & folders
http / https	Create servers & requests
stream / zlib	Streaming data / compression (gzip)
events	EventEmitter
child_process / cluster / worker_threads	Run other processes / multi-core / threads

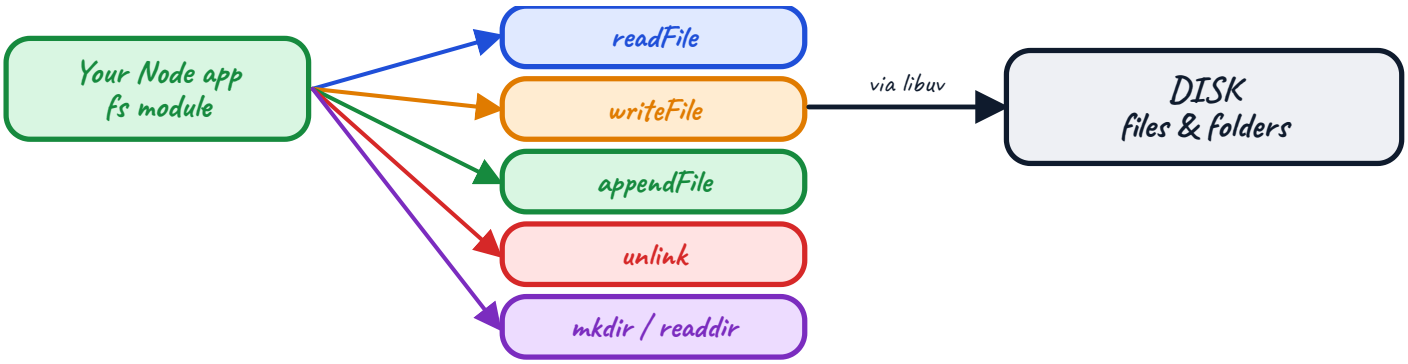
events – EventEmitter (observer pattern)




```
const EventEmitter = require("events");
class Shop extends EventEmitter {}
const shop = new Shop();
shop.on("order", (item, qty) => console.log(`Order: ${qty} x ${item}`));     // listener
shop.once("open", () => console.log("opened once"));                          // only first time
shop.on("error", (e) => console.error("Handled", e.message));                 // always handle "error"!
shop.emit("order", "Pizza", 2);   shop.emit("open");                          // trigger → returns true if listeners
shop.off("order", handler);  shop.listenerCount("order");  shop.removeAllListeners();
```

Listeners run **synchronously** in order added. Emitting **"error"** with no listener crashes the app. Streams, **http.Server** and **process** are all emitters.

7. File System – the fs module



3 styles: callback (fs.readFile) · sync (readFileSync) · promise (fs/promises) ← best

Promise style (recommended): fs/promises + async/await

```
import fs from "node:fs/promises";                    // CJS: const fs = require("fs/promises")
try {
  await fs.writeFile("notes.txt", "Hello Node\n");          // create / OVERWRITE
  await fs.appendFile("notes.txt", "Second line\n");         // add to the end
  const text = await fs.readFile("notes.txt", "utf8");       // without "utf8" you get a Buffer
  console.log(text);
  await fs.rename("notes.txt", "old.txt");   await fs.copyFile("old.txt", "copy.txt");
  await fs.unlink("copy.txt");                              // delete a file
} catch (err) { console.error(err.code, err.message); }     // ENOENT = not found, EACCES = no permission
```

Callback & sync styles

```
const fs = require("fs");
fs.readFile("a.txt", "utf8", (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
const data = fs.readFileSync("a.txt", "utf8");  // blocks!
fs.writeFileSync("b.txt", "hi");
```

Folders

```
await fs.mkdir("a/b/c", { recursive: true });
const files = await fs.readdir("src");              // names
const info = await fs.readdir("src", { withFileTypes: true });
info.filter(d => d.isDirectory());
await fs.rm("a", { recursive: true, force: true });
```

Function	Purpose
readFile / writeFile / appendFile	Read whole file / overwrite / add (flag **"a"**)
unlink / rm / rmdir	Delete file / delete files & folders (recursive) / empty dir
mkdir / readdir / rename / copyFile	Create folder / list / move or rename / copy
stat / access / existsSync	Size, dates, isFile() / check permission / quick exists check
watch / createReadStream	Watch changes / stream big files (next chapter)

JSON file as a mini database + stat + watch

```
const file = path.join(__dirname, "users.json");
const users = JSON.parse(await fs.readFile(file, "utf8"));        // read → object
users.push({ id: 3, name: "Asad" });
await fs.writeFile(file, JSON.stringify(users, null, 2));         // object → pretty JSON
const st = await fs.stat(file);  console.log(st.size, st.mtime, st.isFile());
for await (const ev of fs.watch("src")) console.log(ev.eventType, ev.filename);
```

Remember

Relative paths resolve from where you **start node** (**process.cwd()**), not from the file → always use **path.join(__dirname, "file")**. Use **sync** versions only at startup. **writeFile** overwrites without asking!

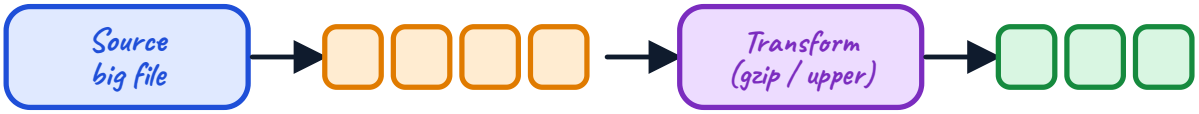
8. Buffers & Streams

Buffer – raw binary data

A **Buffer** is a fixed-size chunk of memory holding **bytes** (files, images, network packets). Like an array of numbers 0–255.

```
const b = Buffer.from("Hello");              // <Buffer 48 65 6c 6c 6f>
b.toString();  b.toString("hex");  b.toString("base64");      // decode  (encodings: utf8, hex, base64)
b.length;  b[0];  Buffer.alloc(10);  Buffer.concat([b, Buffer.from(" Node")]);
Buffer.from("SGVsbG8=", "base64").toString();                  // base64 → "Hello"
```

Streams – process data piece by piece



data flows in small CHUNKS (64 KB) – memory stays tiny, even for a 10 GB file

readable.pipe(transform).pipe(writable) or await pipeline(a, b, c)

Type	Description & example
Readable	Source: ***fs.createReadStream***, HTTP request, ***process.stdin***
Writable	Destination: ***fs.createWriteStream***, HTTP response, ***process.stdout***
Duplex	Both: TCP socket, WebSocket
Transform	Modify while passing: ***zlib.createGzip()***, crypto cipher

Read a big file, copy it, gzip it

```
import fs from "node:fs";  import zlib from "node:zlib";  import { pipeline } from "node:stream/promises";

const rs = fs.createReadStream("big.txt", { encoding: "utf8", highWaterMark: 64 * 1024 });
rs.on("data", chunk => console.log("got", chunk.length));        // events: data, end, error, close
rs.on("end", () => console.log("finished"));  rs.on("error", console.error);

await pipeline(                                                  // handles errors &amp;amp; cleanup (preferred)
  fs.createReadStream("big.txt"),
  zlib.createGzip(),                                             // compress on the fly
  fs.createWriteStream("big.txt.gz"));
```

Stream a video / file to an HTTP response (low memory!)

```
http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "video/mp4" });
  fs.createReadStream("movie.mp4").pipe(res);                    // instead of readFile (loads ALL in RAM)
}).listen(3000);
```

Why streams?

- **Memory efficient** – never load 5 GB at once
- **Fast start** – send first bytes immediately
- **Composable** – pipe small pieces

Backpressure

If the writer is slower than the reader, ***pipe*** / ***pipeline*** automatically pauses the reader (***write()*** returns false → wait for ***"drain"***).

Web Streams (***ReadableStream***) and ***Readable.from(array)***, ***for await (const chunk of stream)*** also work. Typical uses: file upload/download, CSV processing, logging, compression, video.

9. Asynchronous Programming

Async code lets Node start a slow task (file, DB, network) and keep working. Three styles evolved over time:

Callbacks

```
getUser(id, (e,u) => {
getPosts(u, (e,p) => {
  getLikes(p, ...)
})})
"callback hell"
```

Promises

```
getUser(id)
.then(getPosts)
.then(getLikes)
.catch(err)
"chain"
```

async / await

```
const u = await getUser(id)
const p = await getPosts(u)
const l = await getLikes(p)
// try / catch
"reads like sync"
```

1. Callbacks – error-first pattern

```
function readUser(id, callback) {
  setTimeout(() => {
    if (id <= 0) return callback(new Error("Invalid id"));      // 1st argument = error
    callback(null, { id, name: "Asad" });                       // 2nd = result
  }, 500);
}
readUser(1, (err, user) => { if (err) return console.error(err.message); console.log(user); });
```

2. Promises & 3. async / await

```
const readUserP = (id) => new Promise((resolve, reject) =>
  setTimeout(() => (id > 0 ? resolve({ id, name: "Asad" }) : reject(new Error("Invalid id"))), 500));

readUserP(1).then(u => console.log(u)).catch(e => console.error(e.message)).finally(() => console.log("done"));

async function main() {                          // async function ALWAYS returns a promise
  try {
    const user = await readUserP(1);             // pauses THIS function only, not the whole program
    console.log(user.name);
  } catch (e) { console.error(e.message); }
}
main();
```

Promise helper	Behaviour
Promise.all([a,b,c])	Run **in parallel**; resolves when **all** succeed, rejects on the first failure
Promise.allSettled([...])	Waits for all; gives each **{"{status, value|reason}"}**
Promise.race / any	First to settle / first to **succeed**

Sequential vs parallel – big speed difference!

```
const a = await getA();  const b = await getB();          // ✘ 2 s + 2 s = 4 s (one after another)
const [a2, b2] = await Promise.all([getA(), getB()]);     // ✔ about 2 s (together)
import { setTimeout as sleep } from "node:timers/promises";  await sleep(1000);      // await a delay
const readFileAsync = util.promisify(fs.readFile);        // convert callback API to promise
```

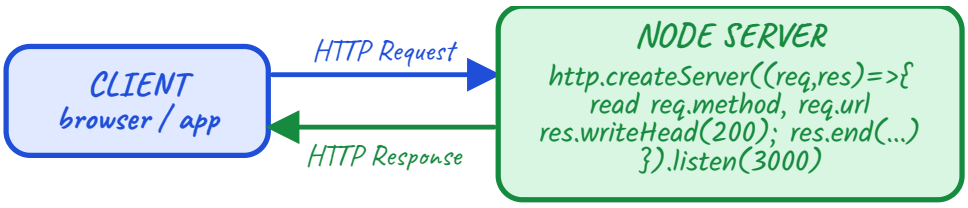
Common mistakes

- Forgetting **await** → you get a pending Promise.
- **await** inside **forEach** doesn't wait – use **for... of** or **Promise.all(map)**.
- Unhandled rejection can crash Node: always **try/catch** or **.catch**.

Under the hood

Promise callbacks go to the **microtask queue** (run before timers); **process.nextTick** runs even before them. **Top-level await** works in ESM.

10. HTTP – building a server without Express



Request: method · url · headers · body

Response: status code · headers · body

server.js – a tiny API with the core http module

```
import http from "node:http";
const users = [{ id: 1, name: "Asad" }, { id: 2, name: "Ali" }];

const server = http.createServer((req, res) => {
  const { method, url } = req;                          // GET, POST ... and "/users"
  res.setHeader("Content-Type", "application/json");

  if (method === "GET" && url === "/") {
    res.writeHead(200, { "Content-Type": "text/html" });
    return res.end("<h1>Welcome to Node server</h1>");
  }
  if (method === "GET" && url === "/users") {
    res.statusCode = 200;  return res.end(JSON.stringify(users));
  }
  if (method === "POST" && url === "/users") {
    let body = "";
    req.on("data", chunk => (body += chunk));           // body arrives in chunks
    req.on("end", () => {
      const user = JSON.parse(body);  users.push(user);
      res.writeHead(201);  res.end(JSON.stringify(user));
    });
    return;
  }
  res.writeHead(404);  res.end(JSON.stringify({ error: "Not Found" }));
});
server.listen(3000, () => console.log("Server on http://localhost:3000"));      // Ctrl+C to stop
```

req (IncomingMessage)
req.method · req.url · req.headers
req.on("data" / "end") – body stream

res (ServerResponse)
res.writeHead(code, headers) · res.statusCode
res.setHeader() · res.write() · res.end(data)

Status	Meaning
200 / 201 / 204	OK / Created / No Content
301 / 302 / 304	Redirect (permanent / temporary) / Not modified (cache)
400 / 401 / 403 / 404	Bad request / Unauthenticated / Forbidden / Not found
500 / 503	Server error / Service unavailable

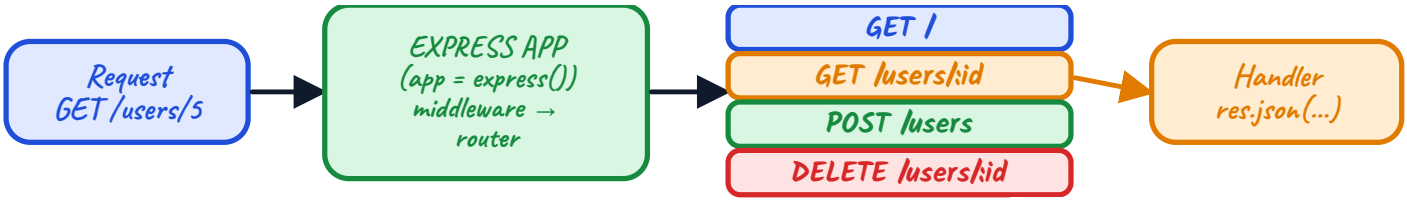
Making an HTTP request (client side in Node 18+)

```
const res = await fetch("https://jsonplaceholder.typicode.com/todos/1");      // built-in fetch
if (!res.ok) throw new Error(res.status);   const todo = await res.json();   console.log(todo.title);
await fetch(url, { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify({ a: 1 })
});
```

Raw http makes routing, body parsing and static files manual → **Express** does it for you. HTTPS: **https.createServer({key, cert}, handler)**.

11. Express.js Basics

Express is the most popular, minimal **web framework** for Node: routing, middleware, easy JSON APIs. ***npm i express***



index.js – first Express app

```
import express from "express";
const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());                          // parse JSON body → req.body
app.use(express.urlencoded({ extended: true }));  // parse HTML form data
app.use(express.static("public"));                // serve files in /public (css, images, html)

app.get("/", (req, res) => res.send("Hello Express 👋"));                    // text / html
app.get("/api/users/:id", (req, res) => {
  const { id } = req.params;                      // /api/users/5  →  "5"
  const { sort, page = 1 } = req.query;           // ?sort=name&page=2
  res.status(200).json({ id, sort, page });       // JSON response
});
app.post("/api/users", (req, res) => {
  const user = req.body;                          // { name: "Asad" }
  res.status(201).json({ message: "Created", user });
});
app.listen(PORT, () => console.log(`Running on http://localhost:${PORT}`));
```

Request (req)	Response (res)
req.params · req.query · req.body	**res.send() · res.json() · res.status(404)**
req.headers · req.cookies* · req.ip	**res.redirect("/") · res.sendFile(path)**
req.method · req.path · req.originalUrl	**res.set() · res.cookie()* · res.download() · res.end()**

HTTP methods ↔ routes

Method	Example	Purpose
GET	**/users, /users/5**	Read data (no body)
POST	**/users**	Create new
PUT / PATCH	**/users/5**	Replace / partially update
DELETE	**/users/5**	Remove

Routers – split routes into files

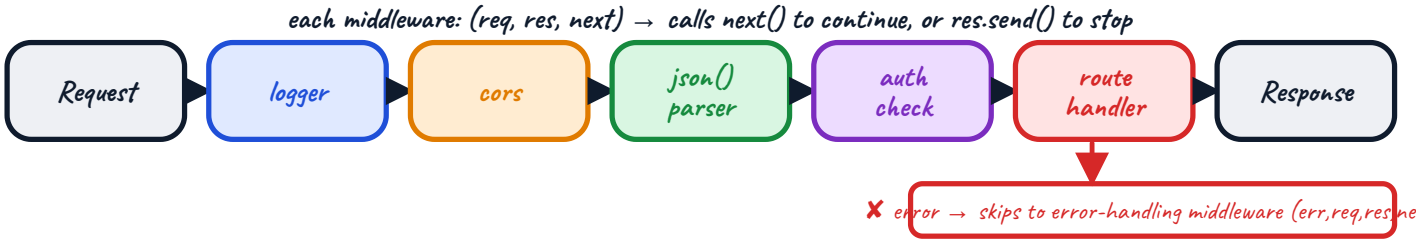
```
// routes/users.js
import { Router } from "express";   const router = Router();
router.get("/", listUsers);  router.get("/:id", getUser);  router.post("/", createUser);
export default router;
// index.js
import userRoutes from "./routes/users.js";   app.use("/api/users", userRoutes);      // prefix for all routes inside
app.route("/book").get(read).post(create);                                           // chain methods on one path
app.all("*", (req, res) => res.status(404).json({ error: "Route not found" }));      // 404 – put LAST
```

Route order matters – first match wins. **Route params** ***:id*** · **query** ***?a=1*** · **body** JSON. Use **nodemon** in dev. Alternatives: **Fastify**, **Koa**, **NestJS**, **Hono**.

12. Middleware & Error Handling

Middleware

A function **(req, res, next)** that runs **between** the request and the final route handler. It can read/modify **req** & **res**, end the response, or call **next()** to pass control on.



Custom middleware examples

```
const logger = (req, res, next) => {                    // 1. logger
  console.log(`${new Date().toISOString()} ${req.method} ${req.url}`);
  next();                                               // without next() the request hangs!
};
const auth = (req, res, next) => {                      // 2. authentication guard
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ error: "No token" });
  req.user = verify(token);  next();                    // attach data for later handlers
};
app.use(logger);                                        // global (every request)
app.get("/profile", auth, (req, res) => res.json(req.user));   // only this route
app.use("/admin", auth, adminRouter);                   // a whole path
```

Type	Examples
Built-in	**express.json() · express.urlencoded() · express.static()**
Third-party	**cors · helmet · morgan (logs) · cookie-parser · compression · express-rate-limit · multer**
Error-handling	**(err, req, res, next) – 4 arguments!**

Error handling

```
app.get("/users/:id", async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) { const e = new Error("User not found"); e.status = 404; throw e; }
    res.json(user);
  } catch (err) { next(err); }                          // forward to error middleware (Express 5 does this for async automatically)
});
app.use((req, res) => res.status(404).json({ error: "Not found" }));   // 404 handler (after all routes)
app.use((err, req, res, next) => {                      // central error handler – LAST, 4 params
  console.error(err.stack);
  res.status(err.status || 500).json({ error: err.message || "Server error" });
});
```

asyncHandler wrapper (Express 4)

```
const asyncHandler = fn => (req, res, next) =>
  Promise.resolve(fn(req, res, next)).catch(next);
app.get("/x", asyncHandler(async (req, res) => {
  res.json(await getData());
}));
```

- Order: **parsers** → **logger** → **routes** → **404** → **errors**.
- Process level: ***process.on("unhandledRejection")***, ***"uncaughtException"*** → log & exit, let PM2 restart.

13. REST API & CRUD

REST

REpresentational State Transfer – a style for web APIs: resources are **nouns in the URL** (**/users**), actions are **HTTP methods**, data is usually **JSON**, each request is **stateless**.



CRUD = Create (POST) · Read (GET) · Update (PUT/PATCH) · Delete (DELETE). Good URLs: **/api/v1/posts/5/comments** (plural nouns, no verbs like /getUsers).

Full CRUD API (in-memory array)

```
import express from "express";
const app = express();  app.use(express.json());
let todos = [{ id: 1, title: "Learn Node", done: false }];
let nextId = 2;

app.get("/api/todos", (req, res) => {                    // READ all (+ filter ?done=true)
  const { done } = req.query;
  res.json(done === undefined ? todos : todos.filter(t => String(t.done) === done));
});
app.get("/api/todos/:id", (req, res) => {                // READ one
  const todo = todos.find(t => t.id === Number(req.params.id));
  if (!todo) return res.status(404).json({ error: "Todo not found" });
  res.json(todo);
});
app.post("/api/todos", (req, res) => {                   // CREATE
  const { title } = req.body;
  if (!title) return res.status(400).json({ error: "title is required" });
  const todo = { id: nextId++, title, done: false };
  todos.push(todo);  res.status(201).json(todo);
});
app.patch("/api/todos/:id", (req, res) => {              // UPDATE (partial)
  const todo = todos.find(t => t.id === Number(req.params.id));
  if (!todo) return res.status(404).json({ error: "Todo not found" });
  Object.assign(todo, req.body);  res.json(todo);
});
app.delete("/api/todos/:id", (req, res) => {             // DELETE
  todos = todos.filter(t => t.id !== Number(req.params.id));
  res.status(204).end();                                 // 204 = success, no body
});
app.listen(3000);
```

Good API practice

Plural nouns, lowercase, version **/api/v1**
Correct **status codes** (201 create, 204 delete, 404, 400, 401)
Validate input (zod / joi)
Pagination **?page=2&limit=10**, sorting, filtering

Test it

Postman / Thunder Client / Insomnia, or terminal:

```
curl -X POST localhost:3000/api/todos -H "Content-Type: application/json" -d '{"title":"Hi"}'
```

14. Template Engines, Forms, Cookies & File Upload

EJS template engine – server-side HTML

server

```
app.set("view engine", "ejs");                          // npm i ejs   → views/ folder is default
app.get("/profile", (req, res) => res.render("profile", { user: { name: "Asad", skills: ["JS", "Node"] } }));
```

views/profile.ejs

```
<h1>Hello, <%= user.name %></h1>                <%# <%= prints escaped value · <%- raw HTML · <% code %>
<ul>
  <% user.skills.forEach(skill => { %>
    <li><%= skill %></li>
  <% }) %>
</ul>
<% if (user.admin) { %> <a href="/admin">Admin</a> <% } %>
```

Others: **Pug**, **Handlebars**. Partials: **<%- include("header") %>**. Modern apps often serve a **React front-end** + JSON API instead.

Handling HTML forms

```
app.use(express.urlencoded({ extended: true }));            // needed for <form method="POST"> data
app.post("/contact", (req, res) => {
  const { name, email, message } = req.body;                // from input name="..." attributes
  if (!email.includes("@")) return res.status(400).send("Invalid email");
  res.redirect("/thanks");                                  // Post → Redirect → Get pattern
});
```

File upload with Multer

npm i multer

```
import multer from "multer";
const storage = multer.diskStorage({
  destination: "uploads/",
  filename: (req, file, cb) => cb(null, Date.now() + "-" + file.originalname),
});
const upload = multer({ storage, limits: { fileSize: 2 * 1024 * 1024 },          // 2 MB max
  fileFilter: (req, file, cb) => cb(null, ["image/png", "image/jpeg"].includes(file.mimetype)) });
app.post("/upload", upload.single("avatar"), (req, res) => {      // <input type="file" name="avatar">
  res.json({ file: req.file.filename, size: req.file.size });     // multiple: upload.array("photos", 5) → req.files
});
// HTML:  <form method="POST" action="/upload" enctype="multipart/form-data">
```

Cookies & sessions

npm i cookie-parser express-session

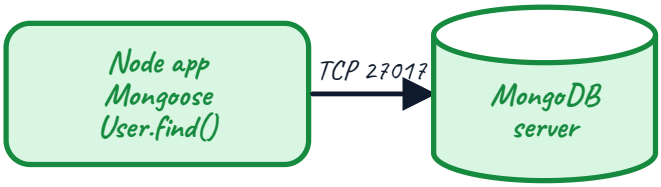
```
app.use(cookieParser());
res.cookie("theme", "dark", { maxAge: 86400000, httpOnly: true, secure: true, sameSite: "strict" });   // set
console.log(req.cookies.theme);   res.clearCookie("theme");                                            // read / delete
app.use(session({ secret: process.env.SESSION_SECRET, resave: false, saveUninitialized: false }));
req.session.userId = user.id;                                   // data kept on the server, browser gets a session-id cookie
```

Other uploads

Store big files on **AWS S3 / Cloudinary** (not on the server disk). Serve uploads with **app.use("/uploads", express.static("uploads"))**. Send email: **Nodemailer**.

15. MongoDB & Mongoose

MongoDB is a **NoSQL document database** storing flexible JSON-like (BSON) documents. **Mongoose** is an **ODM** library that adds schemas, validation and easy queries. Host free on **MongoDB Atlas**. ***npm i mongoose***



Hierarchy

- Database shop
- Collection users (≈ table)
- Document { _id, name, age } (≈ row, BSON/JSON)

db.js – connect

```
import mongoose from "mongoose";
await mongoose.connect(process.env.MONGO_URI);        // mongodb://localhost:27017/shop  or Atlas
mongodb+srv://...
mongoose.connection.on("error", console.error);
```

models/User.js – schema & model

```
const userSchema = new mongoose.Schema({
  name:  { type: String, required: [true, "Name required"], trim: true, minlength: 2 },
  email: { type: String, required: true, unique: true, lowercase: true, match: /^\S+@\S+\.\S+$/ },
  age:   { type: Number, min: 0, max: 120, default: 18 },
  role:  { type: String, enum: ["user", "admin"], default: "user" },
  tags:  [String],
  posts: [{ type: mongoose.Schema.Types.ObjectId, ref: "Post" }],      // relation (reference)
}, { timestamps: true });                                              // createdAt, updatedAt
userSchema.methods.greet = function () { return "Hi " + this.name; };  // instance method
export default mongoose.model("User", userSchema);                      // collection "users"
```

CRUD	Mongoose code
Create	`await User.create({"{name, email}"}) or new User(data).save()`
Read	`User.find() · find({"{age: {$gte: 18}}"}) · findById(id) · findOne({"{email}"})`
Update	`User.findByIdAndUpdate(id, {"{age: 22}"}, {"{new: true, runValidators: true}"})`
Delete	`User.findByIdAndDelete(id) · deleteMany({"{}"})`
Query	`.sort("-createdAt").skip(10).limit(10).select("name email").populate("posts").lean()`

controller using the model

```
export const getUsers = async (req, res, next) => {
  try {
    const { page = 1, limit = 10, q } = req.query;
    const filter = q ? { name: new RegExp(q, "i") } : {};
    const users = await User.find(filter).sort("-createdAt").skip((page - 1) * limit).limit(+limit);
    res.json({ total: await User.countDocuments(filter), users });
  } catch (e) { next(e); }
};
```

Operators

$gt $gte $lt $lte $ne $in $or $and · update: **$set $inc $push $pull** · **aggregate()** pipelines ($match, $group, $sort) · indexes: ***schema.index({"{email:1}"})***.

Remember

Documents get ***_id*** (ObjectId) automatically. Duplicate unique key → error code **11000**. Invalid id → ***CastError***. Embed small data, reference large/shared data.

16. SQL Databases & ORMs

	SQL (MySQL, PostgreSQL)	NoSQL (MongoDB)
Data	Tables, rows, columns, fixed schema	Collections of flexible JSON documents
Relations	Foreign keys, JOINs	Embedding / references
Transactions	Strong ACID, ideal for money & orders	Supported, but design differs
Best for	Structured, related data (banking, e-commerce)	Fast-changing / hierarchical data, prototypes

Using MySQL / PostgreSQL directly (driver)

npm i mysql2 (PostgreSQL: npm i pg)

```
import mysql from "mysql2/promise";
const pool = mysql.createPool({ host: "localhost", user: "root", password: process.env.DB_PASS, database: "shop", connectionLimit: 10 });

const [rows] = await pool.query("SELECT * FROM users WHERE age > ?", [18]);          // ? placeholders = prevents SQL injection!
await pool.execute("INSERT INTO users (name, email) VALUES (?, ?)", ["Asad", "a@x.com"]);
await pool.execute("UPDATE users SET age = ? WHERE id = ?", [22, 1]);
await pool.execute("DELETE FROM users WHERE id = ?", [1]);
```

SQL Injection

Never build queries with string concatenation: **"... WHERE name='" + input + "'"** lets attackers run their own SQL. **Always use parameters (? / $1)** or an ORM.

ORMs – work with objects, not SQL

ORM	Notes
Prisma	Modern, type-safe, schema file + migrations (very popular)
Sequelize	Classic ORM for MySQL/Postgres/SQLite
TypeORM / Drizzle / Knex	TypeScript ORM / lightweight / query builder

Prisma example (npx prisma init → prisma migrate dev)

```
// prisma/schema.prisma:  model User { id Int @id @default(autoincrement())  email String @unique  posts Post[] }
import { PrismaClient } from "@prisma/client";   const prisma = new PrismaClient();
const user = await prisma.user.create({ data: { email: "a@x.com", name: "Asad" } });
const adults = await prisma.user.findMany({ where: { age: { gte: 18 } }, include: { posts: true }, orderBy: { id: "desc" }, take: 10 });
await prisma.user.update({ where: { id: 1 }, data: { name: "New" } });   await prisma.user.delete({ where: { id: 1 } });
```

Other data tools

- **Redis** – in-memory key-value store for **caching**, sessions, rate limits, queues (**npm i redis**). **SQLite** – file database (**better-sqlite3**, built-in **node:sqlite** in new versions).
- **Migrations** = versioned scripts that change the DB schema. **Connection pool** = reuse connections. **Transactions**: **BEGIN ... COMMIT / ROLLBACK** so several queries succeed or fail together. **Index** speeds up searches.

17. Authentication & Authorization

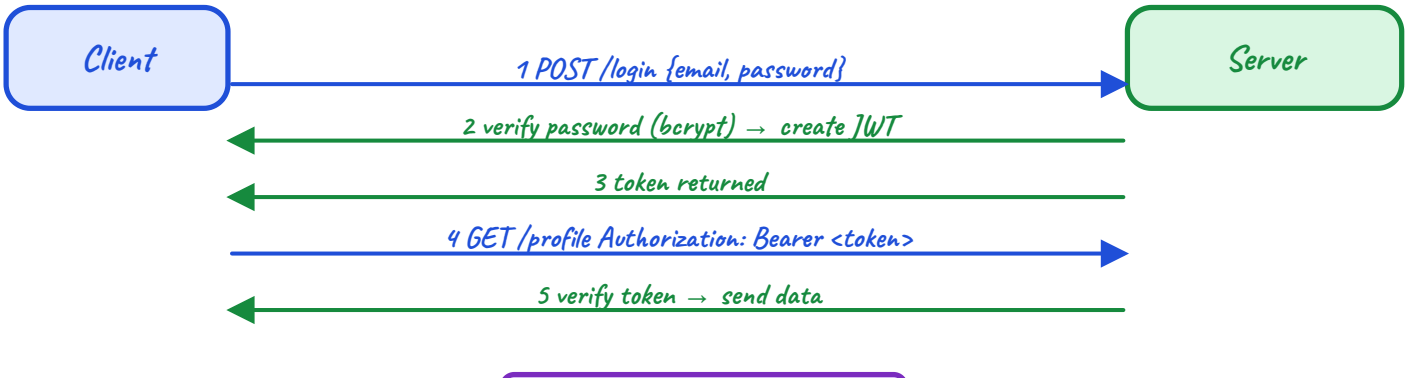
Authentication = *who are you?* (login). **Authorization** = *what may you do?* (roles: user, admin).

Step 1 – never store plain passwords: hash with bcrypt

npm i bcrypt jsonwebtoken

```
import bcrypt from "bcrypt";
const hash = await bcrypt.hash(password, 10);              // salt rounds = 10 → store only the hash
const ok = await bcrypt.compare(password, user.password);  // true / false at login
```

Step 2 – JWT (JSON Web Token) flow



JWT = header . payload . signature

auth routes + middleware

```
import jwt from "jsonwebtoken";
app.post("/api/register", async (req, res) => {
  const { name, email, password } = req.body;
  if (await User.findOne({ email })) return res.status(409).json({ error: "Email already used" });
  const user = await User.create({ name, email, password: await bcrypt.hash(password, 10) });
  res.status(201).json({ id: user._id, email });
});
app.post("/api/login", async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user || !(await bcrypt.compare(req.body.password, user.password)))
    return res.status(401).json({ error: "Invalid credentials" });          // same message for both cases
  const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, { expiresIn: "1h" });
  res.json({ token });
});
export const protect = (req, res, next) => {                                // middleware
  const token = req.headers.authorization?.split(" ")[1];                   // "Bearer <token>"
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ error: "Invalid or expired token" }); }
};
export const restrictTo = (...roles) => (req, res, next) =>
  roles.includes(req.user.role) ? next() : res.status(403).json({ error: "Forbidden" });
app.get("/api/admin", protect, restrictTo("admin"), (req, res) => res.json({ secret: 42 }));
```

- **JWT** is signed (not encrypted): don't put secrets in the payload. Short expiry + **refresh token**.
- Store token in **httpOnly cookie** (safer vs XSS) or memory; ***localStorage*** is easy but XSS-prone.
- **Sessions** = server stores state (cookie holds id) vs **JWT** = stateless token.
- **OAuth / social login**: Passport.js, Auth0, Clerk, NextAuth. Add **2FA**, password reset by email.

18. Security & Environment Variables

Environment variables – keep secrets out of code

.env (add to .gitignore!)

```
PORT=3000
NODE_ENV=development
MONGO_URI=mongodb://localhost:27017/shop
JWT_SECRET=a-very-long-random-string
```

```
import "dotenv/config";                    // npm i dotenv   (Node 20+: node --env-file=.env app.js)
const { PORT = 3000, MONGO_URI } = process.env;        // always strings!   process.env.NODE_ENV === "production"
```

Never commit .env

Commit a **.env.example** with fake values. In production set variables in the host dashboard (Render, Railway, Vercel, Docker).

Security checklist for Express apps

Threat	Protection
Insecure headers	**app.use(helmet())** – sets security HTTP headers
CORS errors / abuse	**app.use(cors({ origin: 'https://myapp.com' }))** allow only your front-end
Brute-force / DDoS	**express-rate-limit**: **rateLimit({ windowMs: 15*60*1000, max: 100 })**
Weak passwords	bcrypt / argon2 hashing, min length, lockout
SQL / NoSQL injection	Parameterised queries; **express-mongo-sanitize**; validate types
XSS	Escape output (EJS **<%= %>**), sanitize HTML (**DOMPurify** / **xss**), CSP header, httpOnly cookies
CSRF	SameSite cookies, CSRF tokens (for cookie auth)
Bad input	Validate & sanitize with **zod / joi / express-validator**; limit body size **express.json({ limit:'10kb'})**
Vulnerable packages	**npm audit**, Dependabot, keep Node updated
Data in transit	**HTTPS** everywhere (TLS), secure cookies, HSTS

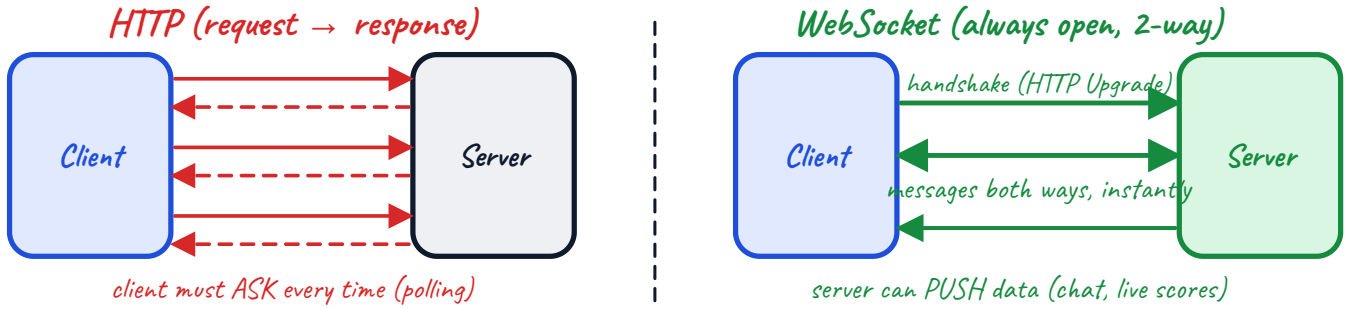
Putting it together

```
import helmet from "helmet";  import cors from "cors";  import rateLimit from "express-rate-limit";  import { z } from "zod";
app.use(helmet());  app.use(cors({ origin: process.env.CLIENT_URL }));
app.use("/api", rateLimit({ windowMs: 15 * 60 * 1000, max: 100 }));
app.use(express.json({ limit: "10kb" }));
const schema = z.object({ email: z.string().email(), password: z.string().min(8) });
app.post("/api/register", (req, res) => {
  const r = schema.safeParse(req.body);   if (!r.success) return res.status(422).json({ errors: r.error.flatten() });
  /* ...create user... */ });
```

- Don't expose **stack traces**, DB errors or **X-Powered-By** (**app.disable("x-powered-by")**, helmet does it).
- Principle of **least privilege** for DB users & API keys.
- Log security events; never log passwords/tokens.
- Follow the **OWASP Top 10**. Use **trust proxy** behind Nginx so rate-limit sees real IPs.

19. Real-time Apps – WebSockets & Socket.IO

Normal HTTP is **request → response**: the server can't talk first. A **WebSocket** is a **persistent two-way connection** – perfect for chat, live notifications, multiplayer games, dashboards, collaborative editing.



Socket.IO (*npm i socket.io socket.io-client*) wraps WebSockets with auto-reconnect, **rooms**, broadcasting and fallbacks. Plain library: **ws**.

server.js – chat server

```
import http from "node:http";  import express from "express";  import { Server } from "socket.io";
const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "http://localhost:5173" } });

io.on("connection", (socket) => {                          // a client connected
  console.log("user connected", socket.id);
  socket.on("join", (room) => socket.join(room));           // rooms = groups
  socket.on("chat message", (msg) => {                       // listen for custom event from client
    io.emit("chat message", { id: socket.id, msg });        // send to EVERYONE
    // socket.broadcast.emit(...) → everyone except sender   io.to(room).emit(...) → one room
    // socket.emit(...) → only this client
  });
  socket.on("disconnect", () => console.log("user left", socket.id));
});
server.listen(3000);
```

client (browser / React)

```
import { io } from "socket.io-client";
const socket = io("http://localhost:3000");
socket.emit("chat message", "Hello everyone!");                    // send
socket.on("chat message", (data) => console.log(data.id, data.msg)); // receive
```

Method	Meaning
socket.emit("event", data)	Send to the other side (client ↔ server)
socket.on("event", fn)	Listen for an event
io.emit / socket.broadcast.emit	All clients / all except the sender
socket.join(room) / io.to(room).emit	Group users and message a group (private chat, game lobby)

Other real-time options

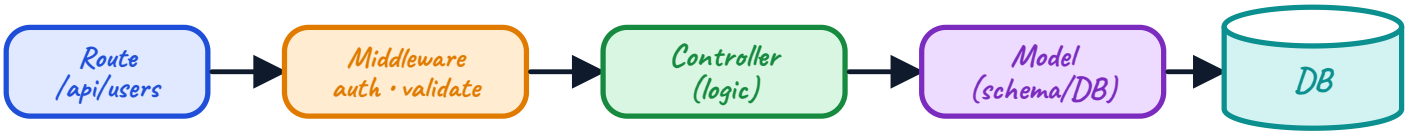
Server-Sent Events (SSE) – one-way server → client over HTTP · **Long polling** · **WebRTC** (video calls) · **MQTT** (IoT).

Scaling note

With many servers, sockets need a shared adapter (**Redis adapter**) and sticky sessions so events reach every client. Authenticate sockets with a token in *io(url, {"{auth:{token}}"})*.

20. Project Structure & MVC Pattern

A clean structure keeps code easy to find, test and grow. **MVC** splits an app into **Model** (data), **View** (output/JSON) and **Controller** (logic). In APIs the "view" is the JSON response.



Response (JSON) ← controller returns data ← model results

project layout

```
my-api/
├── src/
│   ├── config/          db.js  env.js
│   ├── models/          User.js  Post.js
│   ├── controllers/     user.controller.js
│   ├── routes/          user.routes.js
│   ├── middleware/      auth.js  error.js
│   ├── services/        email.service.js
│   ├── utils/           asyncHandler.js
│   ├── app.js           # express app (no listen)
│   └── server.js        # connect DB + listen
├── tests/
├── .env  .env.example  .gitignore
├── package.json  README.md
```

Layer	Job
routes	Map URL + method → controller; attach middleware
controllers	Read req, call services/models, send res
models	Schemas & DB access
services	Business logic (reusable, testable)
middleware	auth, validation, logging, errors
config	Env & database connection

app.js vs server.js (separate so tests can import app)

```
// app.js
const app = express();  app.use(express.json());  app.use("/api/users", userRoutes);  app.use(errorHandler);
export default app;
// server.js
import app from "./app.js";  import connectDB from "./config/db.js";
await connectDB();  app.listen(process.env.PORT, () => console.log("API ready"));
```

controllers/user.controller.js + routes/user.routes.js

```
export const createUser = asyncHandler(async (req, res) => {
  const user = await userService.create(req.body);          // logic lives in the service layer
  res.status(201).json(user);
});
// routes
router.route("/").get(protect, getUsers).post(validate(userSchema), createUser);
router.route("/:id").get(getUser).put(protect, updateUser).delete(protect, restrictTo("admin"), deleteUser);
```

Best practices

- One responsibility per file; keep controllers thin
- Config from **env**; never hard-code secrets
- Central **error handler** & async wrapper
- **Validate** all input; consistent JSON responses
- Use **ESLint + Prettier**, **TypeScript** if possible
- Logging: **pino / winston + morgan**
- Document the API: **Swagger / OpenAPI**, README
- Git: small commits, **.gitignore** node_modules & .env

Also

Feature-based folders (***features/users/...***), dependency injection, **NestJS** enforces this architecture for you.

21. Testing & Debugging

Testing

Level	What & tools
Unit	One function in isolation – **Jest**, **Vitest**, **node:test** (built-in), Mocha + Chai
Integration / API	Routes + DB together – **Supertest** (HTTP calls without starting a port)
End-to-end	Whole app like a user – Playwright, Cypress

```
npm i -D jest supertest          # package.json → "test": "jest"      (ESM: node --experimental-vm-modules)
npm test        npx jest --watch        npx jest --coverage
```

math.test.js – unit test

```
import { add } from "./math.js";
describe("add()", () => {
  test("adds two numbers", () => { expect(add(2, 3)).toBe(5); });
  test("handles negatives", () => { expect(add(-1, -1)).toBe(-2); });
  test("throws on text", () => { expect(() => add("a", 1)).toThrow(); });
});
```

todos.test.js – API test with Supertest

```
import request from "supertest";  import app from "../src/app.js";
beforeAll(async () => await connectTestDB());   afterAll(async () => await mongoose.disconnect());
test("POST /api/todos creates a todo", async () => {
  const res = await request(app).post("/api/todos").send({ title: "Write tests" }).expect(201);
  expect(res.body).toMatchObject({ title: "Write tests", done: false });
});
test("GET /api/todos/999 → 404", async () => { await request(app).get("/api/todos/999").expect(404); });
```

Matchers: **toBe** (===) · **toEqual** (deep) · **toBeTruthy** · **toContain** · **rejects.toThrow**. Mocks: **jest.fn()**, **jest.mock("./db")**, **jest.spyOn**. Use a **separate test DB** (or mongodb-memory-server). **TDD**: write failing test → code → refactor.

Debugging

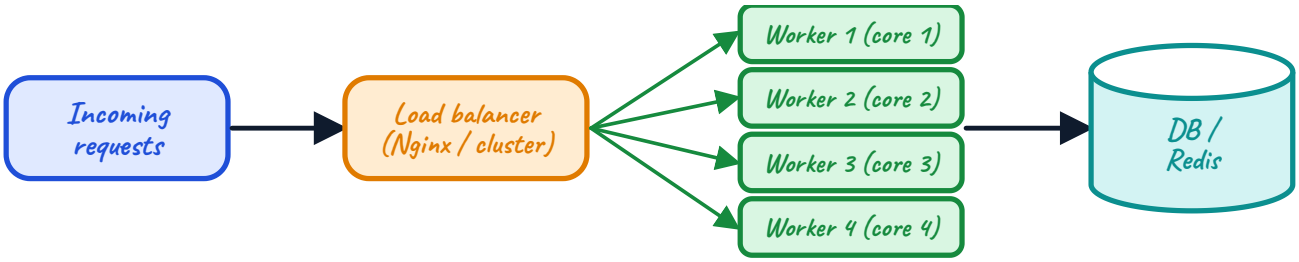
Tool	How
console.log / table / trace	Quick checks (remove later)
VS Code debugger	Run & Debug → set **breakpoints**, step over/into, watch variables, call stack
node --inspect app.js	Open **chrome://inspect** → DevTools for Node (**--inspect-brk** pauses at start)
debugger;	Statement that pauses when an inspector is attached
DEBUG=express:* node app.js	**debug** package – namespaced logs
nodemon / node --watch	Restart on file change

Reading errors

Read the **first line** (type + message) and the **first stack frame in your own file**. Common: ***Cannot find module*** (not installed / wrong path) · ***EADDRINUSE*** (port busy – kill process or change PORT) · ***ENOENT*** (file missing) · ***TypeError: x is undefined*** · ***UnhandledPromiseRejection*** (missing catch) · ***ERR_MODULE_NOT_FOUND*** (ESM needs .js extension).

22. Scaling & Performance

One Node process uses **one CPU core**. To use all cores and handle more users:



1 Node process = 1 CPU core → run one worker per core (cluster / PM2 / containers)

Tool	Use
cluster	Fork one **worker process per CPU core**, all share the same port
worker_threads	Run **CPU-heavy JS** (image processing, big loops) in a separate thread without blocking the loop
child_process	Run other programs / scripts: ***exec, spawn, fork*** (e.g. ffmpeg, python)
PM2	Process manager: ***pm2 start app.js -i max*** (cluster), auto-restart, logs, monitoring

cluster

```
import cluster from "node:cluster";
import os from "node:os";
if (cluster.isPrimary) {
  os.cpus().forEach(() => cluster.fork());
  cluster.on("exit", () => cluster.fork());  // restart
} else {
  app.listen(3000);   // each worker serves
}
```

worker_threads

```
import { Worker, isMainThread, parentPort,
  workerData } from "node:worker_threads";
if (isMainThread) {
  const w = new Worker(new URL(import.meta.url),
{ workerData: 40 });
  w.on("message", r => console.log("fib =", r));
} else parentPort.postMessage(fib(workerData));
```

Performance tips

- **Never block** the loop - use async APIs & streams
- **Cache** repeated results: in-memory (***Map***, node-cache) or **Redis** with TTL
- **Database**: add **indexes**, ***select*** only needed fields, paginate, ***lean()***, connection pool
- **compression()** middleware (gzip) for responses
- Run in ***NODE_ENV=production*** (Express caches views, less verbose)
- Serve **static files via Nginx / CDN**, not Node
- **Profile**: ***node --prof***, ***clinic.js***, ***0x***, Chrome DevTools; **load test** with autocannon / k6
- Use **message queues** (BullMQ + Redis, RabbitMQ) for slow jobs (emails, reports)

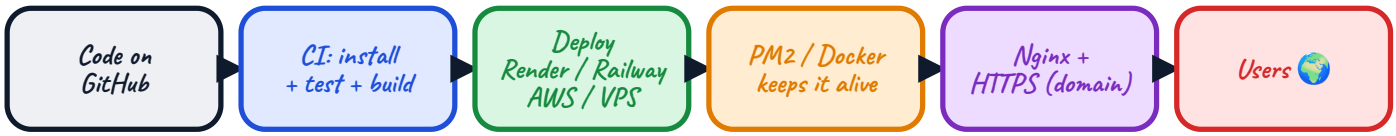
Simple cache middleware

```
const cache = new Map();
const cached = (ttl = 60_000) => (req, res, next) => {
  const hit = cache.get(req.originalUrl);
  if (hit && Date.now() - hit.t < ttl) return res.json(hit.data);
  const send = res.json.bind(res);  res.json = (data) => { cache.set(req.originalUrl, { data, t: Date.now() });
return send(data); };  next();
};
app.get("/api/products", cached(30_000), listProducts);
```

Scale out

Vertical = bigger server · **Horizontal** = more servers behind a **load balancer** (Nginx, AWS ALB). Keep servers **stateless** (sessions in Redis/JWT) so any instance can handle any request. **Microservices**, containers (Docker/Kubernetes), serverless.

23. Deployment



Before you deploy – production checklist

- "start": "node src/server.js" script in package.json
- Listen on process.env.PORT (host assigns it)
- Env variables set on the host (no .env upload)
- NODE_ENV=production; engines.node version
- DB on the cloud (MongoDB Atlas, Neon, Supabase, PlanetScale)
- CORS origin = real front-end URL
- Helmet, rate limit, HTTPS, logging, health route /health
- .gitignore node_modules & .env; commit lock file

Option	How it works
Render / Railway / Fly.io / Koyeb	Connect GitHub repo → build npm install, start npm start → auto-deploys on every push (beginner friendly, free tiers)
Vercel / Netlify functions	Serverless: each route is a function, great for small APIs & Next.js
VPS (DigitalOcean, AWS EC2)	Your own Linux server: install Node, clone repo, run with PM2, put Nginx in front, get free HTTPS with Let's Encrypt (certbot)
Docker / Kubernetes	Package app + Node version in an image → runs the same everywhere (ECS, Cloud Run, Azure)

Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev                # install production deps only (cached layer)
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
CMD ["node", "src/server.js"]
```

VPS commands (PM2 + Nginx)

```
git clone <repo> && cd app && npm ci --omit=dev
pm2 start src/server.js --name api -i max        # cluster mode      pm2 save && pm2 startup    # survive reboot
pm2 logs · pm2 status · pm2 restart api · pm2 monit
# Nginx reverse proxy:  location / { proxy_pass http://localhost:3000; proxy_set_header Host $host; }
sudo certbot --nginx -d api.example.com          # free HTTPS certificate
```

CI/CD with GitHub Actions

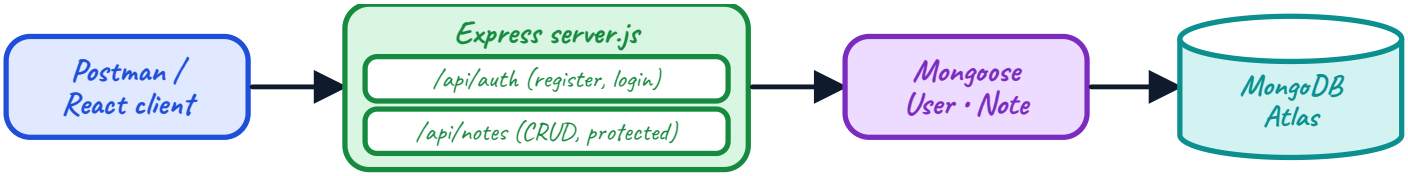
On push: npm ci → npm test → deploy. CI = Continuous Integration (auto build & test), CD = Continuous Delivery/Deployment.

After deploy

Monitor logs & uptime (UptimeRobot), errors (Sentry), metrics (Prometheus / Grafana). Graceful shutdown: handle SIGTERM → server.close().

24. Mini Project – Notes API (Express + MongoDB + JWT)

Features: register / login · JWT protection · create, read, update, delete **your own notes** · validation · error handling. **Uses:** Express, Mongoose, bcrypt, jsonwebtoken, dotenv, cors, helmet.



JWT in Authorization header protects /api/notes · each user sees only THEIR notes

Setup

```
mkdir notes-api && cd notes-api && npm init -y && npm pkg set type=module
npm i express mongoose dotenv bcrypt jsonwebtoken cors helmet && npm i -D nodemon
# package.json scripts → "dev": "nodemon server.js", "start": "node server.js"      .env → PORT, MONGO_URI, JWT_SECRET
```

server.js

```
import "dotenv/config";  import express from "express";  import mongoose from "mongoose";
import cors from "cors";  import helmet from "helmet";
import authRoutes from "./routes/auth.js";  import noteRoutes from "./routes/notes.js";

const app = express();
app.use(helmet());  app.use(cors());  app.use(express.json());
app.use("/api/auth", authRoutes);
app.use("/api/notes", noteRoutes);
app.use((req, res) => res.status(404).json({ error: "Route not found" }));
app.use((err, req, res, next) => res.status(err.status || 500).json({ error: err.message }));      // error handler

await mongoose.connect(process.env.MONGO_URI);
app.listen(process.env.PORT || 3000, () => console.log("Notes API running"));
```

models.js (User + Note)

```
const userSchema = new Schema({
  name: String,
  email: { type: String, unique: true, required: true },
  password: { type: String, required: true },
}, { timestamps: true });
export const User = model("User", userSchema);

const noteSchema = new Schema({
  title: { type: String, required: true },
  body: String,
  owner: { type: Schema.Types.ObjectId, ref: "User", required: true },
}, { timestamps: true });
export const Note = model("Note", noteSchema);
```

middleware/auth.js

```
import jwt from "jsonwebtoken";
export default function protect(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ error: "Login required" });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: "Invalid token" });
  }
}
```

24. Notes API – routes & testing

routes/auth.js

```
const router = Router();
router.post("/register", async (req, res, next) => {
  try {
    const { name, email, password } = req.body;
    if (!email || !password || password.length < 8) return res.status(400).json({ error: "Email and 8+ char
password required" });
    if (await User.findOne({ email })) return res.status(409).json({ error: "Email already registered" });
    const user = await User.create({ name, email, password: await bcrypt.hash(password, 10) });
    res.status(201).json({ id: user._id, email: user.email });
  } catch (e) { next(e); }
});
router.post("/login", async (req, res, next) => {
  try {
    const user = await User.findOne({ email: req.body.email });
    if (!user || !(await bcrypt.compare(req.body.password, user.password))) return res.status(401).json({ error:
"Invalid credentials" });
    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: "1d" });
    res.json({ token });
  } catch (e) { next(e); }
});
export default router;
```

routes/notes.js – every route needs a login

```
const router = Router();   router.use(protect);                                  // protect ALL routes below
router.get("/", async (req, res) => res.json(await Note.find({ owner: req.user.id }).sort("-createdAt")));
router.post("/", async (req, res) => {
  if (!req.body.title) return res.status(400).json({ error: "title required" });
  res.status(201).json(await Note.create({ ...req.body, owner: req.user.id }));      // owner from the TOKEN, not
from the client
});
router.put("/:id", async (req, res) => {
  const note = await Note.findOneAndUpdate({ _id: req.params.id, owner: req.user.id }, req.body, { new: true,
runValidators: true });
  note ? res.json(note) : res.status(404).json({ error: "Note not found" });
});
router.delete("/:id", async (req, res) => {
  const note = await Note.findOneAndDelete({ _id: req.params.id, owner: req.user.id });
  note ? res.status(204).end() : res.status(404).json({ error: "Note not found" });
});
export default router;
```

Try it with curl (or Postman)

```
curl -X POST localhost:3000/api/auth/register -H "Content-Type: application/json" -d
'{"email":"a@x.com","password":"secret123"}'
curl -X POST localhost:3000/api/auth/login    -H "Content-Type: application/json" -d
'{"email":"a@x.com","password":"secret123"}'   # → {"token":"eyJ..."}
curl -X POST localhost:3000/api/notes -H "Authorization: Bearer <token>" -H "Content-Type: application/json" -d
'{"title":"My first note"}'
curl localhost:3000/api/notes -H "Authorization: Bearer <token>"
```

Steps & ideas

1. Build step by step, test each route in **Postman**. 2. Add validation with **zod**, pagination & search. 3. Write **Supertest** tests. 4. Push to **GitHub**, deploy on **Render** with MongoDB Atlas. 5. Connect a **React** front-end.

More ideas: URL shortener · blog API · e-commerce · chat app · file uploader · task manager.

25. Node.js Interview Questions & Answers

1. **What is Node.js?** An open-source, cross-platform JavaScript runtime built on Chrome's V8 engine for running JS outside the browser (servers, tools).
2. **Is Node.js a language or framework?** Neither – it's a **runtime environment**. Express is a framework running on it.
3. **Why is Node fast / scalable?** V8 compiles to machine code and a **non-blocking, event-driven** I/O model handles many connections on one thread.
4. **What is the event loop?** The libuv mechanism that picks completed async callbacks from queues and runs them when the call stack is empty.
5. **Is Node single-threaded?** Your JS runs on one thread, but libuv uses a thread pool (4 default) for fs/crypto/dns, and worker threads / cluster add parallelism.
6. **Blocking vs non-blocking?** Blocking (readFileSync) halts execution until done; non-blocking returns immediately and calls back later.
7. **Order: nextTick, Promise, setTimeout, setImmediate?** sync code → ***process.nextTick*** → promise microtasks → timers → I/O → ***setImmediate***.
8. **What is npm? package.json vs package-lock.json?** Package manager. package.json lists deps & scripts; the lock file pins exact versions for repeatable installs.
9. **dependencies vs devDependencies?** Needed at runtime vs only for development/testing (nodemon, jest).
10. **CommonJS vs ES Modules?** require/module.exports (sync, default) vs import/export (async, static, top-level await, needs ***"type":"module"***).
11. **What is module.exports vs exports?** ***exports*** is a reference to ***module.exports***; reassigning ***exports*** breaks it – assign to module.exports.
12. **What are streams? Types?** Handle data in chunks: Readable, Writable, Duplex, Transform. Use ***pipe/pipeline***; saves memory.
13. **What is a Buffer?** Fixed-size raw binary memory for handling bytes (files, TCP, images).
14. **What is EventEmitter?** Class in ***events*** implementing the observer pattern: ***on, once, emit, off***.
15. **Callback hell & how to avoid?** Deeply nested callbacks → use Promises, async/await, modular functions.
16. **What is middleware in Express?** Function ***(req,res,next)*** run in sequence; can modify req/res, end the cycle or call next().
17. **Application vs router vs error middleware?** app.use global; router-level for grouped routes; error one has 4 params ***(err,req,res,next)***.
18. **req.params vs req.query vs req.body?** Route path variables ***/u/:id*** • URL query ***?a=1*** • payload (needs express.json()).
19. **PUT vs PATCH? PUT vs POST?** PUT replaces whole resource, PATCH partial update; PUT is idempotent, POST creates (not idempotent).
20. **What is REST?** Stateless client-server style using HTTP methods on resource URLs, usually JSON.

25. Interview Q&A (continued)

1. **How to handle errors in Node/Express?** try/catch with async, **`.catch`**, **`next(err)`** to a central error middleware, and process handlers for unhandledRejection/uncaughtException.
2. **What is CORS?** Browser rule blocking cross-origin requests unless the server sends allow headers; fix with the **`cors`** package.
3. **Sessions vs JWT?** Sessions: server stores state, cookie holds id. JWT: signed stateless token the client sends each request.
4. **How do you store passwords?** Never plain text; hash with salt using bcrypt/argon2 and compare at login.
5. **What is JWT structure?** **`header.payload.signature`** (base64url); signed with a secret - payload is readable, not secret.
6. **How to secure an Express app?** helmet, cors, rate-limit, validation, HTTPS, env secrets, parameterised queries, npm audit, httpOnly cookies.
7. **What is `process.nextTick` vs `setImmediate`?** nextTick runs right after current operation (before loop continues); setImmediate runs in the check phase.
8. **What is the cluster module?** Forks worker processes (one per core) sharing a port to use all CPUs.
9. **Worker threads vs child process?** Threads share memory in one process for CPU-heavy JS; child processes are separate OS processes (run other programs).
10. **How to avoid blocking the event loop?** Use async APIs, streams, chunk big loops, offload CPU work to worker threads/queues.
11. **What is libuv?** C library providing the event loop, thread pool and async I/O across operating systems.
12. **What is a callback vs promise vs async/await?** Function passed to run later / object for a future value with then/catch / syntax sugar making promises read synchronous.
13. **What is `Promise.all`?** Runs promises in parallel; rejects if any fails (**`allSettled`** waits for all).
14. **MongoDB vs SQL?** Document vs relational; flexible schema vs fixed schema, joins and ACID.
15. **What is Mongoose?** ODM adding schemas, validation, middleware (hooks), populate and queries on MongoDB.
16. **How to connect Node to DB safely?** Connection string in env, pooling, parameterised queries/ORM, error handling on connect.
17. **What is a reverse proxy / PM2?** Nginx forwards requests to Node & handles HTTPS/static; PM2 keeps the app alive, clusters, restarts and logs.
18. **How do you debug / test Node?** VS Code debugger, **`--inspect`**, console; Jest/Vitest/node:test + Supertest.
19. **What are environment variables?** Config outside code (**`process.env`**, .env + dotenv) for secrets and environment differences.
20. **Explain a request lifecycle in Express.** Request → global middleware (logger, parsers, auth) → route handler → response; errors jump to the error middleware.

Interview tip

Explain with a real flow: "Client sends JWT → Express middleware verifies → controller calls Mongoose → JSON response; errors go to a central handler; I run it with PM2 behind Nginx."

26. Node.js Cheat Sheet

CLI & npm

```
node app.js · node --watch app.js
npm init -y · npm i pkg · npm i -D pkg
npm run dev · npm start · npm test
npm ci · npm audit · npx tool
```

Modules

```
const x = require("./x");  module.exports = {a};
import x, { a } from "./x.js";  export default f;
import fs from "node:fs/promises";
```

fs & path

```
await fs.readFile(p, "utf8");  fs.writeFile(p, d);
fs.appendFile(p, d);  fs.unlink(p);  fs.mkdir(p,
{recursive:true});
path.join(__dirname, "a", "b.txt");
```

Async

```
async function f(){ try{ await g() }catch(e){} }
await Promise.all([a(), b()]);
setTimeout(fn, ms);  process.nextTick(fn);
```

Events & streams

```
em.on("e", fn); em.emit("e", data);
fs.createReadStream(a).pipe(fs.createWriteStream(b));
```

Express

```
const app = express();  app.use(express.json());
app.get("/u/:id", (req, res) => res.json(req.params));
app.post("/u", (req, res) => res.status(201).json(req.bo
app.use("/api", router);  app.use(express.static("public"
app.use((err,req,res,next)=>res.status(500).json({error:
app.listen(3000);
```

Mongoose

```
await mongoose.connect(uri);
const M = mongoose.model("U", new Schema({ name:
M.create(d) · M.find(q) · M.findById(id)
M.findByIdAndUpdate(id, d, {new:true}) · M.findByIdA
```

Auth & security

```
await bcrypt.hash(pw, 10);  await bcrypt.compare(pw
jwt.sign({id}, SECRET, {expiresIn:"1h"});  jwt.verify(t,
app.use(helmet()); app.use(cors()); rateLimit(...)
```

Status codes

200 OK · 201 Created · 204 No content · 301/302 Redirect · 400 Bad
403 Forbidden · 404 Not found · 409 Conflict · 422 Invalid · 429 Too m
error

Golden rules

Never block the event loop · handle every error · validate all input · secrets in .env · hash passwords · use async/await · small modules · log & monitor · deploy behind HTTPS.

🚀 Build APIs, break things, fix them – that is how you master Node.js! 🚀 — asadokz18